

146. Ranger sa fonction dans un fichier voisin : #include "outils.cpp"

La première sorte d'#include, avec des guillemets : bande() déménage dans l'onglet outils.cpp, et principal.cpp le verse chez lui.

1. Ce qu'il faut comprendre

Ce qui est nouveau ici : la fonction a déménagé. Elle n'est plus dans `principal.cpp` : elle est dans un **second onglet**, `outils.cpp`. Dans l'atelier, c'est le bouton « + fichier », au-dessus de l'éditeur, qui crée un onglet comme celui-là.

`#include "outils.cpp"` veut dire « verse ici tout le texte d' `outils.cpp` ». Avant de compiler, la console remplace cette ligne par le contenu du fichier, mot pour mot. Le compilateur voit donc **exactement** le programme de l'étape d'avant : même cartouche, même écran.

Les guillemets "..." désignent un fichier À TOI, écrit à côté du programme. Retiens-le bien, car l'étape 15.6 montrera l'autre sorte, avec des chevrons `<...>`, qui désigne une fonction **de la console**.

Où placer la ligne : l'habitude est de mettre tous les `#include` en haut, ensemble. Pour des **fonctions**, l'ordre ne compte pas : le compilateur relève toutes les fonctions avant de les traduire, et `main()` peut appeler `bande()` même si son texte est collé plus bas. Pour des **variables globales** ou des **dessins**, si : ils doivent être versés **avant** la fonction qui s'en sert.

Pourquoi l'extension .cpp et pas .h : les deux marchent ici. Par habitude, un `.h` (« header ») contient des déclarations (des variables, des noms) ; un `.cpp` contient du code, des fonctions. `outils.cpp` contient une fonction.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **949 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage — la mise en route de la console
bande()	38 o	← main() — une fonction fournie par le langage
main()	64 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AdresseCase	10 o	← poser() — trouve l'adresse d'une case de l'écran
CopierTuiles	18 o	← démarrage — copie les lettres et les dessins en mémoire vidéo

```

EffacerCarte      14 o ← démarrage – vide les deux cartes de l'écran
Tuiles           704 o ← CopierTuiles – la police entière
Lettres gravées : toutes (poser() reçoit un numéro calculé)
Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, EffacerCases, EcrireTexte, EcrireNombre,
Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond,
EffacerPanneau, LireManette, DonneesTransfert, Notes
Fonctions du langage appelées :
  poser() × 1      17 o sur place, dans bande() + AdresseCase (10 o)
  image() × 1      3 o sur place, dans main() + AttendreImage (28 o)
Tes fonctions appelées :
  bande() × 3      57 o sur place, dans main()

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

// Le changement : bande() n'est plus écrite ici, mais dans outils.cpp.
//
// #include "outils.cpp"
// |          |
// |          +-- le fichier à verser : l'onglet « outils.cpp », à côté
// +----- « verse ici » : avant la compilation, cette ligne est
//           remplacée par tout le texte d'outils.cpp. Tout se passe
//           comme si bande() était écrite ici, comme à l'étape d'avant.

#include <poser>      // pose une tuile (c'est bande() qui s'en sert)
#include <ALPHABET>   // les lettres de la police
#include "outils.cpp" // NOUVEAU : mes fonctions à moi, versées ici

```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```

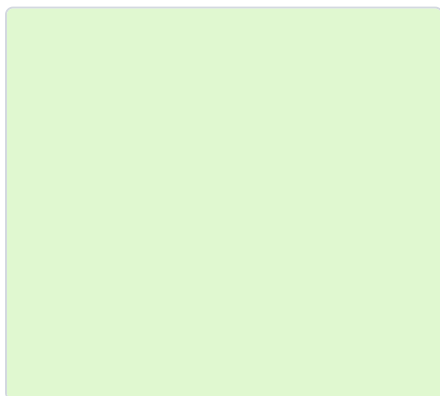
int main() {
  bande(2, 5, ALPHABET[0], 10); // bande() vient d'outils.cpp
  bande(2, 7, ALPHABET[1], 6);
  bande(2, 9, ALPHABET[2], 3);

  while (true) {
    image();
  }
}

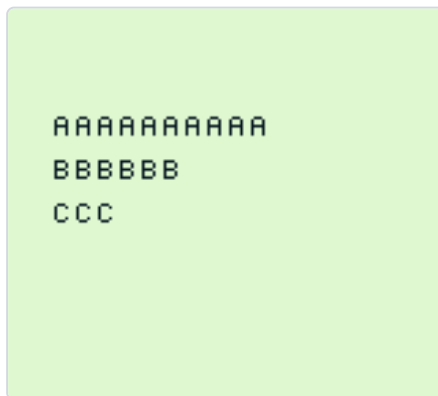
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de commencer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Exactement l'écran d'avant : dix A, six B, trois C. Mais le programme est rangé en deux onglets.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ `bande()` vient d'`outils.cpp` : dix A sur la ligne 5

✓ six B, trois C : le même écran qu'avant

Ensuite : **147. Le fichier voisin écrit ses propres `#include` <...>** — chapitre 25 : Tes propres `#include` : ajouter une fonction. ·
gameboy3 — écrire une cartouche Game Boy en C++